IS F311 - Assignment II
Frog Skeleton Animation - Report

By

Tanveer Singh *2020A7PS0084P*
Varun Sahni *2020A7PS0144P*

Instructions to run the code

- Running the code requires Node.JS as well as NPM installed. Node.JS can be installed from https://nodejs.org/en.
- Open the project folder. There are two dependencies, and they can be installed by typing the following commands *in the current project folder:*

```
# three.js
npm install --save three

# vite
npm install --save-dev vite
```

- Type "npx vite" in the terminal. In a few seconds, the terminal will give the following result:

```
VITE v4.3.3  ready in 362 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h to show help
```

- Now the website can be viewed from http://localhost:5173/ .

Model Development

The frog skeleton has been modeled using basic geometric shapes available in Blender - cylinders, cubes, and torus.

- The first bone we modeled was the **spine**. To do this, we made a torus with a large radius, then removed a major chunk of the torus via **Edit Mode + Remove Vertices.** After that, we made finer modifications to the spine to get a better shape, using **Push/Pull/Extrude Vertices** options in Edit mode in Blender.

- After this, we designed the legs. Only designing one leg is enough, as then it can be duplicated and **reflected** along the necessary axis to produce the other leg. For leg design, long cylinders represent the major bones such as **femur, astragalus, calcaneum, tibiofibular**. The joints such as the knee and ankle have been represented by small cylinders. Representing the joints this way makes hierarchical modeling and animations easier. See **Fig. 1** for the development snapshot at this stage.
- After legs, the arms (actually front legs) were designed in a similar fashion.
- Finally, the skull region was designed. A distorted cylinder was used to make the **parietal** bone. For the **jaw** bones, a torus was used, to capture the curve in the jaws. See **Fig. 2** for a screenshot of the jaws being modeled using a torus.
- The prepared Blender model was exported in **GLTF** format, which is the recommended way of importing models in Three.JS.

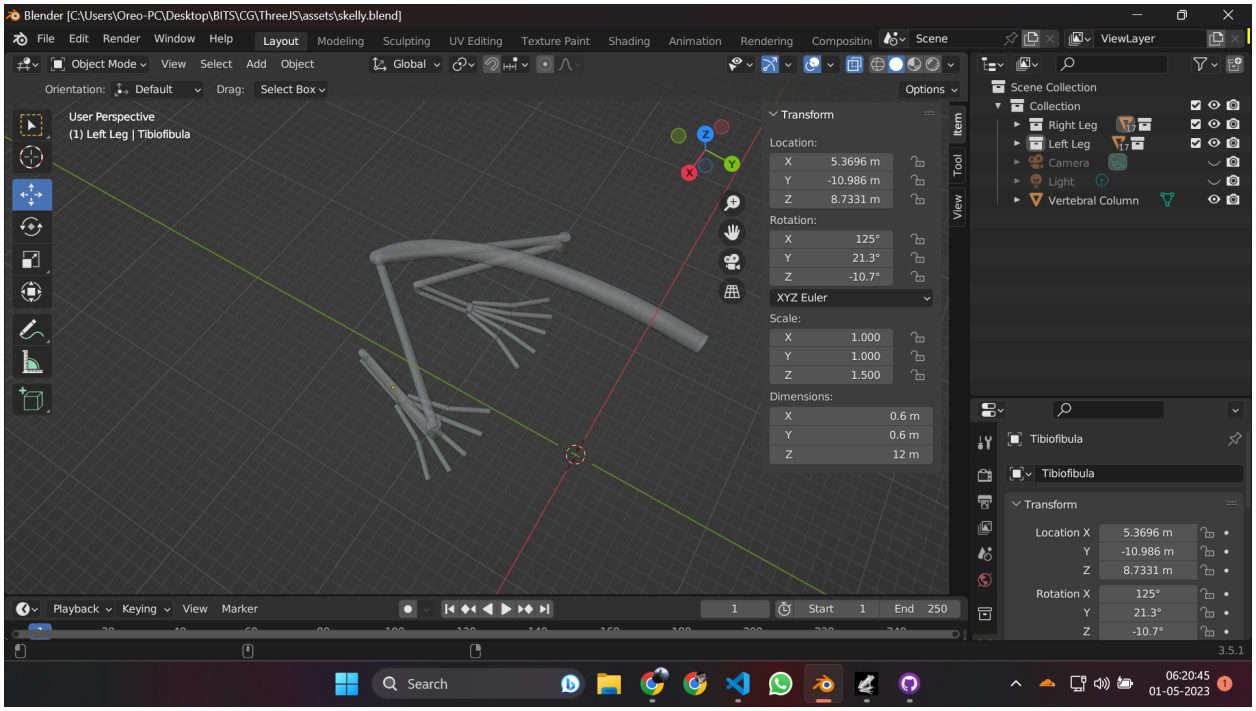

Fig. 1: Development of skeleton - Spine + Legs

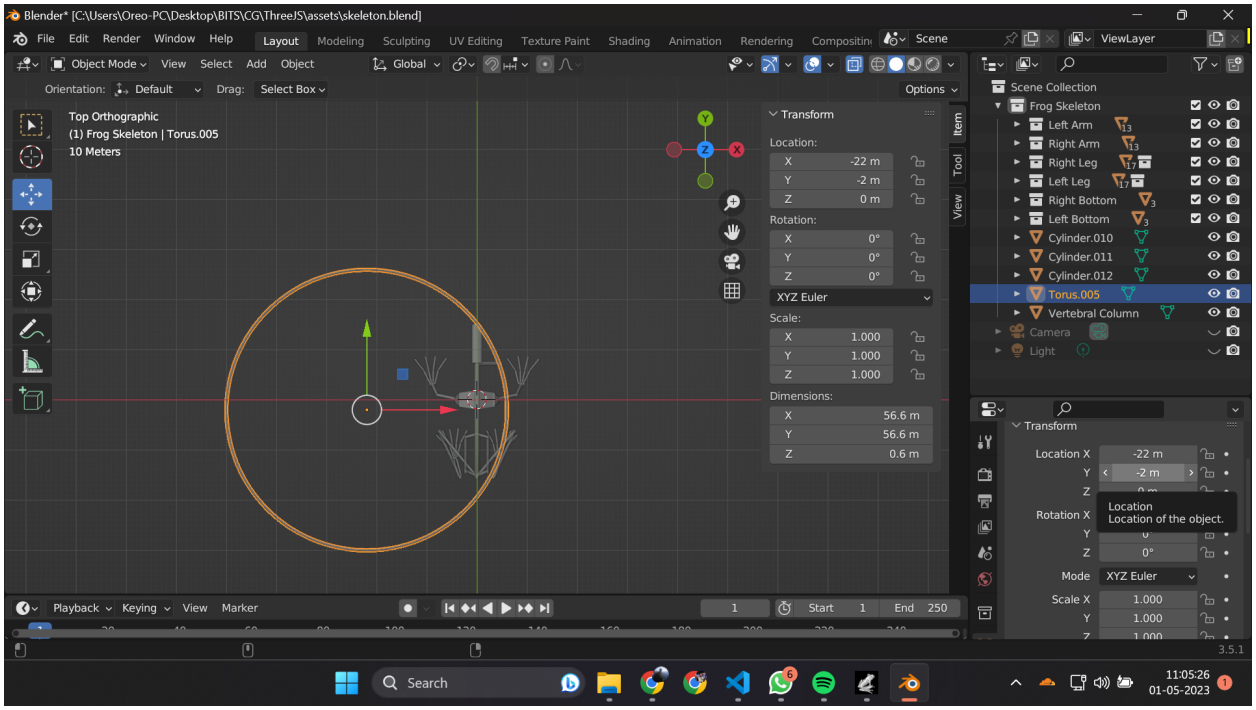

Fig. 2 : Development of skeleton - Jaws using Torus

Hierarchical Modeling

- Although this issue was encountered later in the project, it is worth mentioning at this point as it was fixed in Blender itself.

On importing the GLTF model in Three.JS, we printed the skeleton's object hierarchy on the console (using the *dumpObject* function present in *main.js)*. This did not show any hierarchy at all! There was just one parent object, and all components were children of this object. To fix this, we had to create **empty objects** in Blender, and group together bones as children of that object. **Fig. 3** will show the difference between the old modeling (using Collections in Blender) and new modeling (using Empty Objects to represent parents).

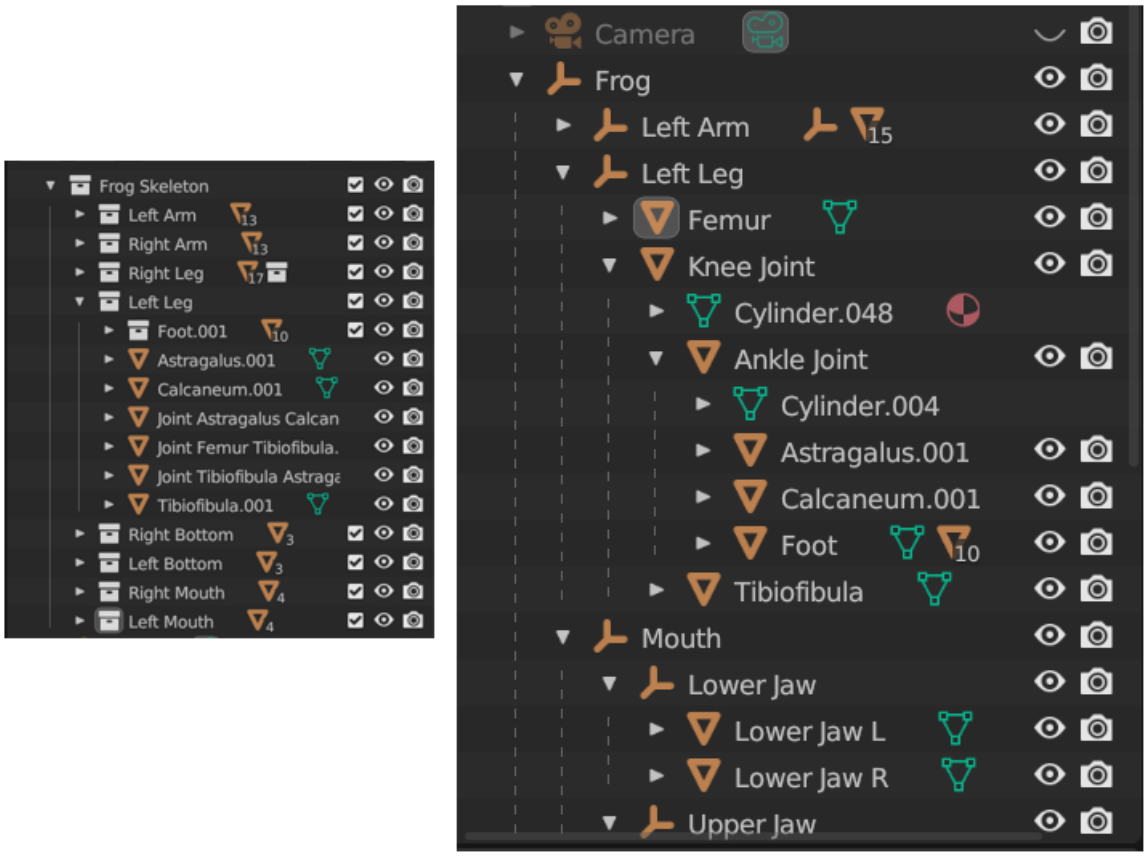

Fig.3 : (L) Old hierarchical modeling, (R) New hierarchical modeling

The final hierarchical structure of the model can be seen on opening the console while reloading the webpage. It is also available in the file *skeleton_hierarchy.txt* in the submitted folder.

Three.JS

We created a basic HTML webpage with only one canvas, and a JS script attached to it. The bulk of code lies in *main.js*. The code is well-commented and divided into sections based on the functionality.

Core website functionality

- First, we create the 3 essential components of any Three.JS website - **renderer**, **camera** and **scene**. We also add **Orbit Controls** for navigation on the website. The controls allow navigation in the following way:
  - **Left Mouse Button** rotates the mouse around the frog
  - **Middle Mouse Button** zooms the scene in or out.
  - **Shift + Left Mouse Button** can be pressed to *pan*.

- Next, we add a simple **ground** to the scene, so that there is some reference object for the frog. (For example, the frog's jumps are calibrated based on the ground). The ground is a simple green checkerboard pattern.
- We have some helper code to print an objects hierarchy, and to resize renderer to display size (so that the website works well on any resolution and screen size).
- At the end, there is a render loop that updates the animation on every frame.

Animations

- The animations are triggered by **events,** which can be recognized in JS using **Event Listeners.**
- Some animations happen on pressing the key, some happen on holding the key. Both types of animations have been implemented, merely to showcase each type of feature.

Core Features

1. **Translation and Rotation**

The **arrow keys** (up, down, left, right) **translate** the object in the corresponding direction. Shift + Arrow keys **rotate** the object about the corresponding axis. The implementation for this is simple and only involves translating or rotating the parent object, via, for example, **obj.translateX** or **obj.rotateZ** functions.

2. **Looking left/right**

Pressing A and D makes the frog look left and right, respectively. The frog's head tilts up to a specified angle (60 degrees).

3. **Moving legs and arms**

Pressing S moves the back legs backward. **Hierarchical modeling** plays a key role in this. First, we rotate the entire leg about the **hip joint.** Then, we rotate the lower leg about the **knee joint.** Then, only the foot rotates, about the **ankle joint.** The various rotations of the joint can be seen in the animation - the leg is moving back, but at the same time, the calf is stretching and the toes are flexing!
Pressing W moves the front legs forward in a similar fashion.
Note: For back legs, most of the movements happen "together" to give a more realistic effect of legs moving. For arms, the movements happen one-by-one, to showcase hierarchical modeling in detail.

Bonus Features

4. **Swimming**

Swimming was the most challenging part of the project! The X key can be pressed for swimming animation. Frogs swim using **breaststroke.** This stroke involves **three stages** of movement for arms:

- Transforming arms from sitting position to swimming position
- Moving arms backward in a circular motion
- Getting arms back to the front, keeping them under the body

For **legs**, the stroke has been slightly simplified. There are **two stages** of movement for legs:

- Transforming legs from sitting position to swimming position
- Moving both legs in an oscillating motion, and in complementary directions

The implementation of each of the stages can be seen in the code (each of them is in a separate function).

5. **Jumping**

The J key can be pressed for jumping animation. Frog jumping also happens in three stages:

- The legs stretch and the frog moves up slightly
- The actual jump
- The legs contract and the frog moves back to ground position

The first and third stages are the reverse of each other, and have been implemented in the same function. The frog translates upwards, and the legs stretch downwards by changing the rotations of each of the joints. For the second stage, the translation velocity is computed at every frame, and the jump has been implemented as if under a **downward force** such as gravity.

Extra Features

We have implemented some extra features and animations.

6. **Mouth open/close**

On pressing the L key, the frog's mouth opens and closes in an alternating fashion. The jaw opening stops at a certain point (when angle is 22.5 degrees).

7. **Reverse front legs movement**

Pressing Shift + W, performs the exact reverse of W. The joints come back to their initial state.

8. **Toggle ground visibility**

Pressing G can remove/add the ground from/to the scene.

9. **Reset all transformations**

To reduce the hassle of refreshing the page a lot, we have added a command "R", which resets all transformations so far, and essentially brings the frog to its initial state.

Side Note : Importance of applying transformations in Blender

When we originally imported the GLTF model from Blender, some of the bones were extremely distorted. We found that the issue was due to **scaling.** Basically, when an object is scaled, say to 2x size, and then its child is added in Blender, there is no issue, the child has the original scale only. But, when loading it in Three.JS, the child's size is doubled! This issue can exponentially scale the objects in case of a deep hierarchy. **Fig. 4** shows the hilarious result we got - a *flying frog!*

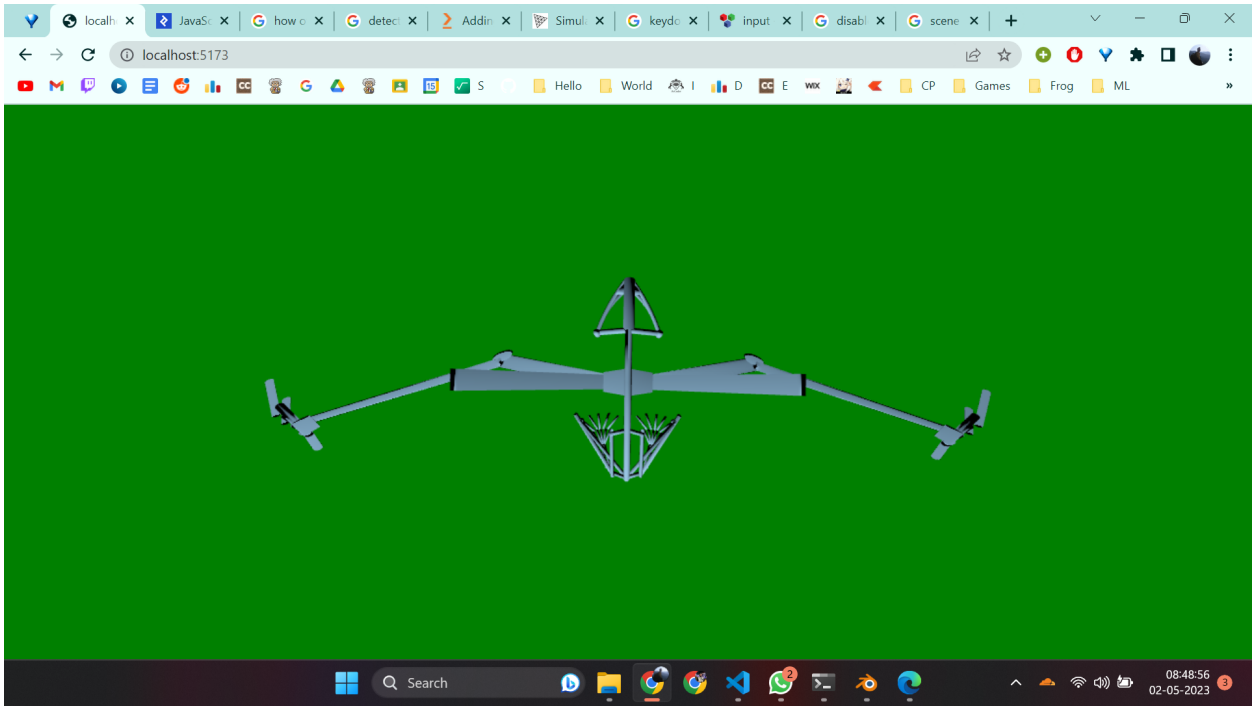

Fig. 4 : A frog with wings!

We fixed the issue by correctly "applying" all the transformations in Blender, and re-exporting the model.

Lighting & Shaders

The **lighting** we have implemented is a **Directional Light.** This is a realistic lighting effect, and the result is similar to sunlight on a scene in real life. The light source is on top of the frog. So, on loading the model, the top of the frog is illuminated, but the bottom portion is dark as there is no light falling on it.

- While an **Ambient Light** would have made the model look better, it would not be a realistic lighting effect, hence we didn't apply it.

Our **shader** has various properties, and we have added a **GUI Panel** on the top-right corner of the screen to see them in effect.

- The **default shader** loads with a blue-ish material.
- The **emissive shader** is a **Phong Shader** with an emissive blue light. This creates a beautiful gradient effect on the arms and legs. The white light is coming from the Directional Light source, and the blue light is being emitted from the fragments themselves. In particular, the top portion is fully white and the bottom is fully blue, with transitions along the sides.
- The **neon wave** shader is a rather fun shader that is updated on every render frame, based on the current time. The code for this Shader is in the *index.html* page. The way a time

component has been added, is by adding **sin(u_time)** to the **color mix**. This creates a wave-like effect.

- The **shadow** effect is a basic **Phong Shader** that captures shadows - the top part is grayish-white and the bottom part is black as there is no light reaching it.

Screenshots of the model

The remaining figures give various screenshots of the model.

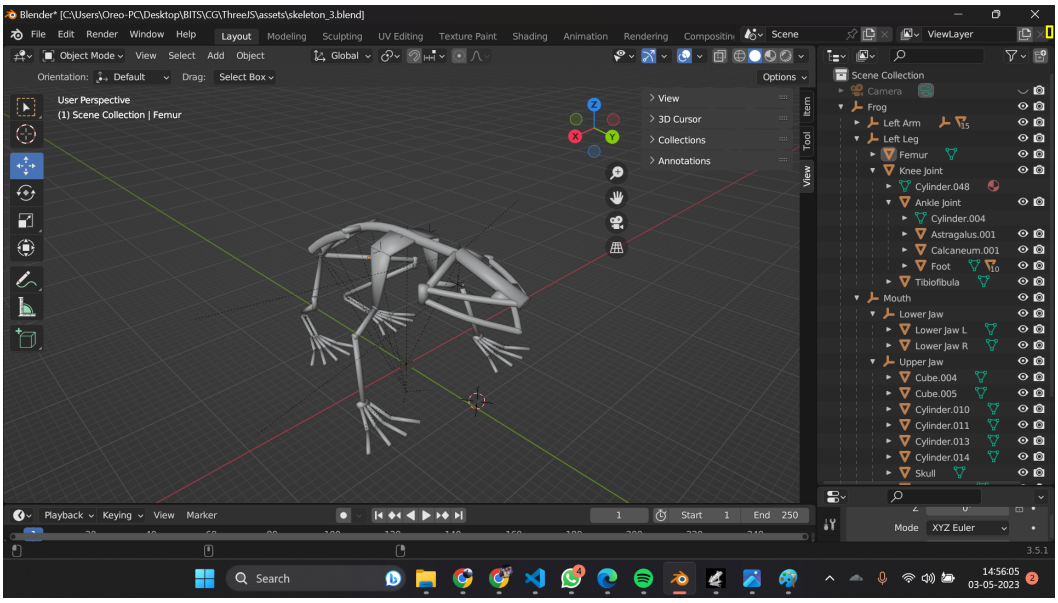

Fig. 5: Final Blender Model

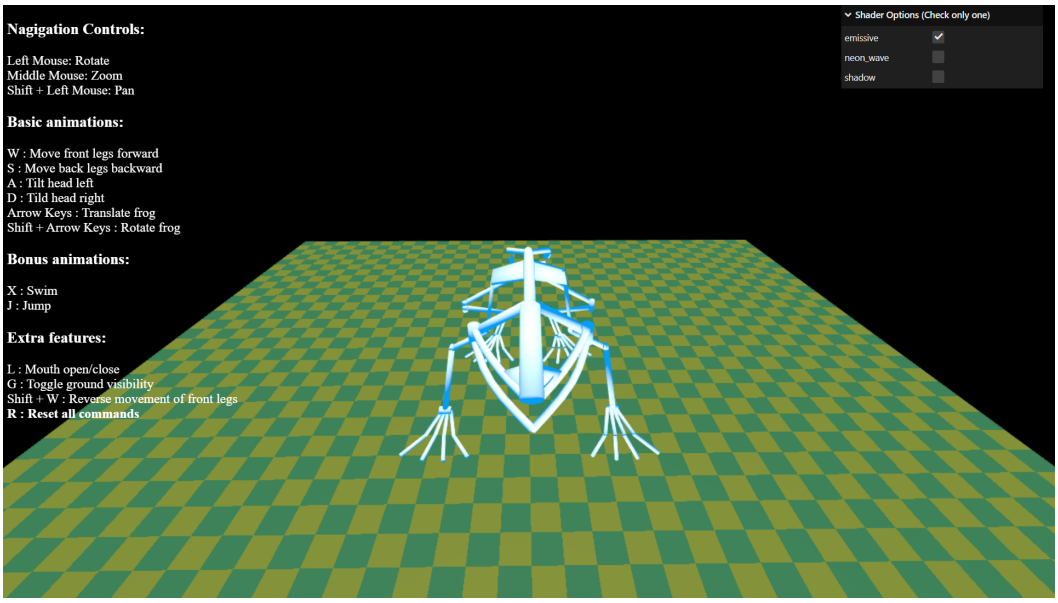

Fig. 6: Final Three.JS Canvas

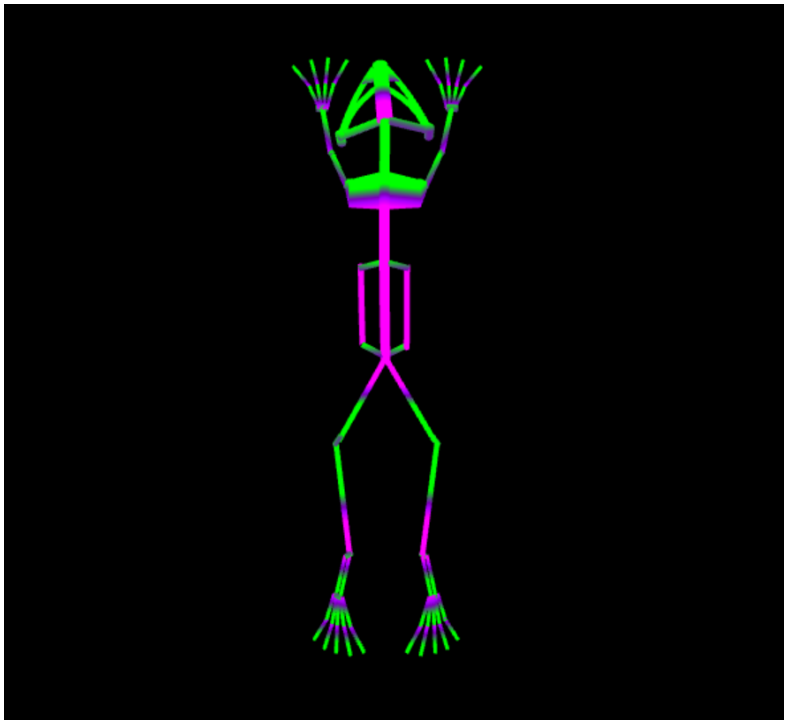

Fig. 7: Frog stretching pose

Fig. 8: Leaping frog